

ASSIGNMENT NO:16.2

Name:Saisharan

Ht.no:2303A51434

bt.no:21

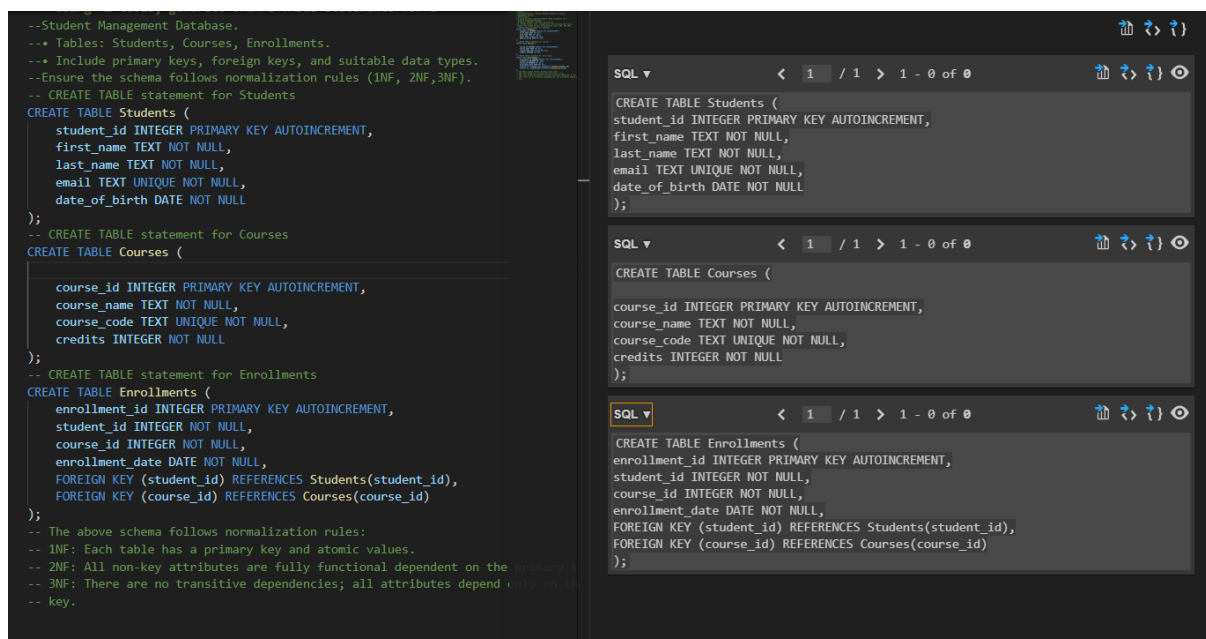
Task Description-1: (Design Database Schema for Student Management System)

Instructions:

- Using AI tools, generate CREATE TABLE statements for a Student Management Database.
- Tables: Students, Courses, Enrollments.
- Include primary keys, foreign keys, and suitable data types.

Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Code & Output screenshot:



```
--Student Management Database.
--* Tables: Students, Courses, Enrollments.
--* Include primary keys, foreign keys, and suitable data types.
--Ensure the schema follows normalization rules (1NF, 2NF,3NF).
-- CREATE TABLE statement for Students
CREATE TABLE Students (
  student_id INTEGER PRIMARY KEY AUTOINCREMENT,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  date_of_birth DATE NOT NULL
);
-- CREATE TABLE statement for Courses
CREATE TABLE Courses (
  course_id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_name TEXT NOT NULL,
  course_code TEXT UNIQUE NOT NULL,
  credits INTEGER NOT NULL
);
-- CREATE TABLE statement for Enrollments
CREATE TABLE Enrollments (
  enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_id INTEGER NOT NULL,
  course_id INTEGER NOT NULL,
  enrollment_date DATE NOT NULL,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
-- The above schema follows normalization rules:
-- 1NF: Each table has a primary key and atomic values.
-- 2NF: All non-key attributes are fully functional dependent on the
-- 3NF: There are no transitive dependencies; all attributes depend on a key.
```

```
SQL
CREATE TABLE Students (
  student_id INTEGER PRIMARY KEY AUTOINCREMENT,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  date_of_birth DATE NOT NULL
);

SQL
CREATE TABLE Courses (
  course_id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_name TEXT NOT NULL,
  course_code TEXT UNIQUE NOT NULL,
  credits INTEGER NOT NULL
);

SQL
CREATE TABLE Enrollments (
  enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_id INTEGER NOT NULL,
  course_id INTEGER NOT NULL,
  enrollment_date DATE NOT NULL,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Task Description- 2: (Insert Sample Records)

Instructions:

- Use AI tools to generate INSERT statements to populate the tables.
- Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Code & Output screenshot :

The screenshot shows a SQL IDE with a code editor on the left and a results pane on the right. The code editor contains SQL statements to insert data into three tables: Students, Courses, and Enrollments. The results pane displays the output of three separate SQL queries: a SELECT * FROM Courses query, a SELECT * FROM Students query, and a SELECT * FROM Enrollments query.

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);

Select * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Task Description- 3: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record
- Validate the changes using SELECT statements.

Code & output screenshot:

The screenshot shows a SQL IDE with a code editor on the left and a results pane on the right. The code editor contains SQL statements to update a student's email, delete an enrollment record, and validate the changes using SELECT statements. The results pane displays the output of four separate SQL queries: an UPDATE Students query, a DELETE FROM Enrollments query, a SELECT * FROM Courses query, and a SELECT * FROM Students query.

```
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
DELETE FROM Enrollments
WHERE enrollment_id = 5;

SELECT * FROM Courses;
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
DELETE FROM Enrollments
WHERE enrollment_id = 5;

SELECT * FROM Courses;
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

student_id	name	email
1	Arjun	arjun_new@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Task Description- 4: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

- Count number of students enrolled per course
- Display courses with more than one student enrolled

Analyze the correctness of results.

Code & output screenshot:

The screenshot displays a SQL IDE interface with two queries and their corresponding results.

Query 1:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

Result 1:

course_name	student_count
Data Structures	1
Database Systems	2
Operating Systems	1

Query 2:

```
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Result 2:

course_name
Database Systems

Task Description- 5: (Query Optimization)

Instructions:

- Use AI tools to analyze query performance.
- Rewrite inefficient sub queries using JOINS and test both queries for correctness

The screenshot displays a SQL IDE interface with a query and its result.

Query:

```
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
SELECT s.name, c.course_name
```

Result:

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com